

# FOOD DELIVERY DATABASE SYSTEM

## Swiggy / Uber Eats Inspired Backend System

### Project Report

Prepared By: \_\_\_\_

**Name: Sabik**

**Course: \_\_\_\_**

**Institution: \_\_**

**Submission Date: \_\_\_\_**

---

## Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. System Requirements
5. ER Diagram
6. Database Schema Design
7. Constraints and Rules
8. Stored Procedures
9. Functions
10. Triggers
11. Transactions
12. Performance Tuning
13. Analytics Layer
14. System Workflow
15. Test Cases
16. Conclusion

---

## 1. Introduction

The Food Delivery Database System is designed to simulate a real-world online food ordering platform similar to Swiggy and Uber Eats. The system manages customer orders, restaurants, food items, delivery partners, payments, and order tracking. PostgreSQL is used as the backend database with SQL and PL/pgSQL features.

---

## 2. Problem Statement

Modern food delivery platforms require efficient order management, payment handling, partner assignment, and tracking systems. The goal of this project is to build a structured and automated database system that supports these operations.

---

## 3. Objectives

- Store customer information
  - Manage restaurants and food items
  - Process customer orders
  - Assign delivery partners
  - Track delivery status
  - Handle payments
  - Generate reports and analytics
  - Improve performance through optimization
- 

## 4. System Requirements

### Software Used

- PostgreSQL
- pgAdmin
- Python (Faker library)

### Languages

- SQL
  - PL/pgSQL
- 

## 5. ER Diagram

Paste your ER diagram image here.

Caption: Figure 1: Entity Relationship Diagram of Food Delivery Database System

---

## 6. Database Schema Design

Tables created:

- Customers
- Orders
- OrderItems
- FoodItems
- Restaurants
- Payments
- DeliveryPartners
- Order\_Status\_History

Relationships:

- One customer can place many orders
- One order can contain multiple food items
- One restaurant contains many food items
- One order has one payment record
- One delivery partner can deliver multiple orders

Insert screenshot of created tables.

Caption: Figure 2: Database Tables Created

---

## 7. Constraints and Rules

Implemented constraints:

### **NOT NULL**

Prevents missing values.

### **UNIQUE**

Used for email and phone.

### **CHECK Constraints**

Examples:

- quantity > 0
- total\_amount >= 0
- status validation

- payment validation

### **DEFAULT Constraints**

Examples:

- order\_date = NOW()
  - payment\_status='pending'
- 

## **8. Stored Procedures**

Implemented procedures:

### **place\_order()**

Creates a new order.

### **add\_order\_item()**

Adds food items into order.

### **assign\_partner()**

Assigns available delivery partner.

### **process\_payment()**

Processes payments.

### **update\_order\_status()**

Updates delivery progress.

---

## **9. Functions**

### **get\_total\_order\_amount()**

Returns total amount of a specific order.

### **customer\_order\_count()**

Returns total order count for customers.

---

## 10. Triggers

### **trg\_subtotal**

Calculates subtotal automatically.

### **trg\_order\_total**

Updates total amount.

### **trg\_order\_status\_tracking**

Tracks order history.

### **trg\_partner\_update**

Updates partner availability.

### **trg\_payment\_update**

Updates payment status.

Insert screenshot.

Caption: Figure 3: Trigger Execution Result

---

## 11. Transactions

Transactions maintain consistency.

Used: BEGIN COMMIT ROLLBACK

Workflow: Order Creation → Add Item → Payment → Commit

---

## 12. Performance Tuning

Performance improvements:

- Index creation
- Query optimization
- EXPLAIN ANALYZE

- Join optimization

Indexes used:

- orders(customer\_id)
- orders(status)
- orderitems(order\_id)
- payments(order\_id)

Insert screenshot of EXPLAIN ANALYZE.

Caption: Figure 4: Query Execution Plan

---

## 13. Analytics Layer

Implemented analytics:

- Most active customers
- Delivery performance
- Order status distribution
- Daily sales trends
- Restaurant revenue

Insert analytics screenshot.

Caption: Figure 5: Analytics Query Result

---

## 14. System Workflow

Customer Login ↓ Place Order ↓ Add Food Items ↓ Calculate Subtotal ↓ Update Total ↓ Assign Partner ↓ Process Payment ↓ Update Status ↓ Track Order History

---

## 15. Test Cases

Insert screenshots:

Figure 6: Sample Customer Data

Figure 7: Order Processing Output

Figure 8: Final Order Status Output

---

## 16. Conclusion

The Food Delivery Database System successfully simulates a real-world backend architecture using PostgreSQL. The project includes automation, business logic, analytics, and optimization techniques for efficient order processing and management.